



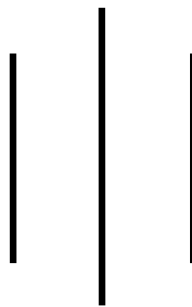
LA GRANDEE INTERNATIONAL COLLEGE

Simalchaur, Pokhara, Nepal

A Project Report

On

“Real-Time Chat Box”



Submitted to

Bachelor of Computer Application (BCA) Program

In partial fulfilment of the requirements for the degree of BCA under

Pokhara University

Submitted by:

Name:	Course	Semester	P.U. Registration Number
Mausam Pariyar	BCA	8 th	2021-1-53-0060
Sagar Poudel	BCA	8 th	2021-1-53-0077
Ashish Gurung	BCA	8 th	2021-1-53-0050

Date: 2025-07-07

Acknowledgement

We have presented this report focusing on the topic “**Real-Time Chat Box**”. This report has been prepared for partial fulfilment of the requirement for degree of BCA and to have practical experience.

We are heartily thankful to the faculty of BCA, **LA Grandee International College** and our supervisor **Mr. Sunil Sapkota** for his role to motivate and lead for this report. We are obliged to him for his constant guidance, supervision and feedbacks which enabled us to prepare a well-executed report.

Further, we express our gratitude to LA Grandee family, classmates, seniors and teachers who have directly and indirectly supported us during our report.

Student's Declaration

"Real-Time Chat Box"

We hereby declare that we are the only authors of this work and that no sources other than those mentioned here have been used in this. We assure you that the work we present here is unique to ourselves and resemblances to another similar project are purely coincidental.

.....

Mausam Pariyar (PU Exam Roll no): 2021-1-53-0060

.....

Sagar Poudel (PU Exam Roll no): 2021-1-53-0077

.....

Ashish Gurung (PU Exam Roll no): 2021-1-53-0050

Program: BCA, 8th Semester

Date: 2025-07-07

Supervisor's Declaration

I hereby recommend that the project entitled **Real-Time Chat Box** done under my supervision by **Mausam Pariyar, Sagar Poudel and Ashish Gurung** during their 8th semester in partial fulfilment of the requirement for the degree of **Bachelor of Computer Application (BCA)** under affiliation of **Pokhara University** is completed to my satisfaction and should be processed for final evaluation.

Mr. Sunil Sapkota

(Project Supervisor)

Date: 2025-07-07

Certificate of Approval

This is to certify that **Mausam Pariyar, Sagar Poudel and Ashish Gurung**, the student of 8th Semester at LA GRANDEE International College, with PU registration number **2021-1-53-0060, 2021-1-53-0077 and 2021-1-53-0050**, under the BCA department, has successfully completed their project. We have examined the report and hereby, approve their work as per the requirement for the partial fulfilment of the degree of Bachelor of Computer Application under Pokhara University.

.....
Mr. Kundan Chaudhary

BCA Coordinator

.....
Er. Amit Parajuli

External Examiner

.....
Er. Kiran K.C

Principal

Date: 2025-07-07

Abstract

Real-Time Chat Box is a mobile and web-based chat application built to make communication within small organizations easier, faster, and more reliable. Whether it's one-on-one conversations or group discussions, the app offers real-time messaging, audio/video calling, and media sharing, all in a clean and easy-to-use interface. With WebSocket technology at its core, messages are delivered instantly, keeping conversations smooth and uninterrupted.

The app is designed to help teams communicate better, avoid common miscommunications, and stay connected — all of which contribute to improve the Organizational Health and Performance (OHP). It's especially useful for small teams that need a simple but effective communication tool to support their day-to-day tasks and overall goals.

On the tech side, the project uses Flutter for the mobile app, React.js for the web interface, Node.js for the backend, and MongoDB as the database. This combination allowed us to build a responsive, real-time application that works well across devices and handles user data securely.

Throughout the development process, we focused on creating something practical, user-friendly, and aligned with real-world communication needs. This project not only helped us apply what we've learned about full-stack development, but also gave us hands-on experience with real-time communication, user interface design, and team collaboration.

Developed as part of the BCA program at Pokhara University, Real-Time Chat Box is more than just a technical project — it's a reflection of how software can bring people together and help organizations communicate better every day.

Table of Contents

Acknowledgement	ii
Student's Declaration	iii
Supervisor's Declaration.....	iv
Certificate of Approval	v
Abstract.....	vi
Table of Contents	vii
List of Tables	ix
List of Figures.....	x
Abbreviation.....	xi
1. Introduction.....	1
2. Problem Statement.....	2
3. Objective.....	4
4. Background Study.....	5
5. Requirement Document	6
6. Methodology.....	12
6.1. Agile Model	12
6.2. Work Assigned.....	15
7. System Design	18
7.1. ER Diagram.....	18
7.2. Dataflow Diagram.....	19
7.3. UML Diagram.....	23
8. Gantt Chart.....	24
9. Development Process.....	25
10. Testing.....	28
11. Deliverables	37
12. Project Result.....	38

13.	Future Enhancements.....	40
14.	Conclusion	42
15.	Reference	43
16.	Annex.....	45

List of Tables

Table 5.1 - Requirement Matrix.....	11
Table 6.1 – Worked Assigned.....	17
Table 8.1 - Gantt Chart	24
Table 10.1 Test case 1.....	29
Table 10.2 Test case 2.....	29
Table 10.3 Test case 3.....	30
Table 10.4 Test case 4.....	31
Table 10.5 Test case 5.....	32
Table 10.6 Test case 6.....	33
Table 10.7 Test case 7.....	34
Table 10.8 Test case 8.....	35
Table 10.9 Test case 9.....	36

List of Figures

Figure 6.1- Agile Model.....	12
Figure 7.1 - ER Diagram.....	19
Figure 7.2 - DFD Level 0.....	20
Figure 7.3 - DFD Level 1.....	21
Figure 7.4 - DFD Level 2.....	22
Figure 7.5 - UML Diagram.....	23
Figure 16.1 - Sign In and Sign Up Page	46
Figure 16.2 – Homepage.....	47
Figure 16.3 – Profile page and Other Settings	48
Figure 16.4 Custom Theme Palette.....	49
Figure 16.5 Chat Screen.....	50
Figure 16.6 Call Screen.....	51

Abbreviation

Acronym	Full Form
API	Application Programming Interface
BCA	Bachelor of Computer Application
CRUD	Create, Read, Update, Delete
DB	Database
DFD	Data Flow Diagram
ER	Entity Relationship
FCM	Firebase Cloud Messaging
GUI	Graphical User Interface
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JWT	JSON Web Token
LAN	Local Area Network
MVC	Model View Controller
NoSQL	Non-relational Structured Query Language
OHP	Organizational Health and Performance
OTP	One-Time Password
REST	Representational State Transfer
SDK	Software Development Kit
SDLC	Software Development Life Cycle

Acronym	Full Form
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
VS Code	Visual Studio Code
WebRTC	Web Real-Time Communication
SQL	Structured Query Language

1. Introduction

In today's digital world, fast and spontaneous communication has become a key driver of collaboration, productivity, and user engagement. Whether in workplaces, customer service environments, or social settings, the need for real-time communication is greater than ever. To address this growing demand, our project, *Real-Time Chat Box*, presents a mobile-based solution designed to enable instant, reliable, and user-friendly messaging for individuals and organizations.

Real-Time Chat Box is an Android-based application that facilitates seamless real-time interaction. It is specifically designed to cater to the internal communication needs of small organizations, where streamlined collaboration and effective information exchange are critical for maintaining Organizational Health and Performance (OHP). The system aims to replace outdated, fragmented communication tools with a centralized platform that ensures faster and more efficient conversations.

The application leverages modern technologies such as WebSocket for real-time data exchange and Flutter for mobile development, offering a responsive and comfortable chatting experience. It includes core features like instant message delivery, secure user authentication, group chats, and multimedia sharing. These functionalities are presented within an intuitive user interface that suits users of varying technical backgrounds.

While the system currently focuses on meeting basic real-time communication needs, it also includes provisions for API integration, opening possibilities for future connectivity with third-party services. This ensures that the platform remains adaptable to evolving communication demands.

The main objective of this project is to eliminate delays in communication, reduce dependency on outdated messaging platforms, and increase organizational productivity through optimized digital interaction. Just as automated systems in other domains aim to improve efficiency such as restaurant systems that streamlines order processing. Real-Time Chat Box seeks to automate and enhance internal communication, making it more efficient, organized, and productive.

2. Problem Statement

In today's digital world, real-time communication is essential for individuals, organizations, and academic projects. However, many popular messaging platforms do not fully meet the needs of small teams or student groups who want a lightweight, secure, and customizable chat application.

Apps like WhatsApp, Telegram, Slack, and Discord are widely used but each has limitations:

- WhatsApp focuses on personal messaging and lacks strong API support and enterprise features.
- Telegram offers bot integration and APIs but lacks advanced user role management.
- Slack is a powerful collaboration tool but can be cost-prohibitive and complex for small or educational projects.
- Discord provides excellent voice and video chat with multimedia support but is often seen as informal and less suitable for structured workflows.

These platforms are mostly proprietary and offer limited customization, making it difficult for developers or students to tailor them to specific needs. Also, their reliance on large backend infrastructure increases complexity and cost, which can be a barrier for lightweight or experimental applications.

Common challenges with existing messaging solutions include:

- Latency issues causing delayed message delivery and reducing the real-time experience.
- Overly complex user interfaces that can negatively impact user adoption.
- Limited extensibility and difficulty integrating with other systems via APIs.
- Inconsistent support for multimedia content such as images, videos, and documents.
- Weak authentication mechanisms, raising concerns about data security and user privacy.

To address these gaps, this project develops Real-Time Chat Box, an Android-based application that utilizes WebSocket protocol for efficient, low-latency communication. It features a clean and responsive UI/UX design, secure authentication, and multimedia sharing capabilities. The system is designed to be modular, allowing API integration for future enhancements.

The goal is to provide a cost-effective, scalable, and open solution that empowers small organizations and academic users to improve internal communication and collaboration without relying on large-scale commercial platforms.

3. Objective

The major goals of this project are as follows:

- To develop a real-time chat application that enables instant messaging, audio/video calls, and multimedia sharing using modern technologies like WebSocket.
- To enhance user experience by designing an intuitive and responsive interface that is easy to use for people with different levels of technical knowledge.
- To optimize communication efficiency and security within an organization by implementing secure user authentication, fast message delivery, and reliable data handling.

4. Background Study

In today's world, real-time chat has become an important part of many applications. It helps improve user engagement, allows instant support, and makes communication faster and easier. This background study looks into how real-time chat systems work, what technologies are used behind them, and where they are commonly applied. It also highlights the gaps in current solutions and explains why there's a need for a better, more user-friendly real-time chat application.

Right now, there are different tools and platforms that offer real-time messaging, such as:

- **Messaging Apps like WhatsApp, Telegram, and Facebook Messenger:** These are great for personal communication but don't offer much support for integrating with business systems.
- **Customer Support Chatbots such as Intercom, Zendesk, and Drift:** These are mostly used in businesses to handle customer queries automatically, but they sometimes lack the human touch needed in more complex situations.
- **In-App Chat SDKs like Firebase, Socket.IO, and Twilio:** These give developers the tools to add chat features to their apps, but they usually require a lot of coding and setup to work properly.

During our research, we also noticed some recent trends:

- **High Demand for Instant Messaging:** People expect quick replies in areas like online shopping, education, health services, and social media.
- **AI-Powered Chatbots Are Growing:** Many companies use chatbots to answer basic questions, but there's also a shift toward combining bots with human support for better service.
- **Focus on Security and Privacy:** More apps now use end-to-end encryption to protect users' messages, like in the Signal app.

Based on this study, we found that while there are many existing options, most of them are either too complex, not flexible enough, or lack proper support for small organizations or student-level projects. That's why we decided to develop our own

Real-Time Chat Box, which is simple, secure, and effective for small teams or academic use.

5. Requirement Document

This requirement documentation outlines the functional, non-functional requirements and the tools and technologies required for the **Real-Time Chat Box**.

5.1. Functional Requirements

- **User Registration and Authentication**

Users can sign up and log in securely using email and password. Authentication is required to access chat features. (Uses secure backend token-based auth, e.g., JWT)

- **One-to-One Messaging**

Users can send and receive real-time text messages to and from other users using WebSocket protocol (via Socket.IO).

- **Typing Indicator**

Users can see when the other person is typing, giving a more interactive experience.

- **Seen Status / Read Receipts**

Messages will have status indicators like sent, delivered, and seen to improve chat clarity.

- **Audio/Video Calling**

Users can make real-time audio and video calls to other users through integrated WebRTC or supported video call APIs.

- **File and Media Sharing**

Users can send multimedia (images, documents, videos) within chat messages.

- **User Profile Management**

Each user has a profile with name, email, and profile picture that can be viewed and updated.

- **Online/Offline Status**

Users can view which of their contacts are online or offline in real time.

- **Chat History and Storage**

Past conversations are stored in the database (MongoDB) and can be accessed anytime.

5.2. Non-Functional Requirements

- **Cross-Platform Compatibility**

The app should work smoothly on both Android (via Flutter) and web (via React.js frontend).

- **Performance**

Real-time message delivery must be fast and responsive, with minimal latency using WebSockets.

- **Security**

- Use token-based authentication (e.g., JWT).
- Messages and user data should be encrypted during transmission.
- Prevent unauthorized access and protect user privacy.

- **Responsive UI/UX**

Simple and intuitive user interface suitable for both technical and non-technical users.

- **Code Maintainability**

Follow clean architecture practices (e.g., separation of concerns, modular structure) to make the codebase easy to update and debug.

- **Low Bandwidth Optimization**

Ensure chat functionality works well even in low internet conditions (fallback messages, retry mechanisms).

- **Data Consistency**

Ensure synced messages across sessions and devices, using MongoDB for reliable storage and retrieval.

Tools and Technologies

Development Tools

- **IDE:** Visual Studio Code
- **Version Control:** Git
- **Repository Hosting:** GitHub
- **Backend Development:** Node.js with Express.js
- **Database:** MongoDB (NoSQL, document-based database)
- **Real-time Communication:** Socket.IO for WebSocket-based real-time messaging
- **API Testing & Debugging:** Postman

Frontend

- **Mobile App:** Flutter (Dart) – Used to build cross-platform mobile UI for Android
- **Web App:** React.js with JavaScript and Tailwind (for dashboard or testing frontend features)
- **UI Design & Styling:** Flutter Widgets, Material Design, and custom UI components

State Management

- **Flutter:** GetX (for routing, state, and socket-based reactive updates)
- **Web:** Zustand or React Context API

Package Managers

- **Flutter:** pub.dev package manager (for packages like get, socket_io_client, image_picker, etc.)
- **Node.js Backend:** npm (for installing backend dependencies like express, mongoose, cors, socket.io, etc.)

Libraries and Dependencies

- **Flutter Packages:**
 - **get:** For GetX state management
 - **socket_io_client:** For WebSocket communication
 - **flutter_webrtc:** For real-time audio/video calling
 - **emoji_picker_flutter:** For emoji support in chat

- **image_picker, file_picker:** For media and document attachments
- **Backend Libraries (Node.js):**
 - **express:** For server setup
 - **mongoose:** MongoDB object modeling
 - **jsonwebtoken:** For secure user authentication
 - **bcryptjs:** For password hashing
 - **cors, dotenv, socket.io**

Other Tools

- **Postman:** For testing and debugging REST APIs
- **MongoDB Atlas and Compass:** Cloud-based NoSQL database service used for hosting MongoDB
- **MS Word:** For documentation and report writing
- **MS PowerPoint:** For preparing presentation slides for project defense

Requirement Matrix

Requirement ID	Requirement Description	Type	Priority	Status
FR-01	Users can securely register and log in using email and password	Functional	High	Completed
FR-02	Users can send and receive real-time messages (one-to-one)	Functional	High	Completed
FR-03	Users can join and participate in group chats	Functional	Medium	In Progress
FR-04	Users can send and receive multimedia content (images, documents, etc.)	Functional	High	Completed
FR-05	Users can make real-time audio and video calls	Functional	High	In Progress
FR-06	Users can view online/offline status of other users	Functional	Medium	Completed
FR-07	Users can see message status (sent, delivered, seen)	Functional	Medium	Completed
FR-08	Users can view and update their personal profile (name, picture, etc.)	Functional	Medium	Completed
FR-09	Users can see typing indicators in real-time	Functional	Medium	Completed
FR-10	App shows chat history and preserves messages on relaunch	Functional	High	Completed
NFR-01	Real-time communication should be low-latency using WebSocket (Socket.IO)	Non-Functional	High	Completed

NFR-02	The UI must be responsive, clean, and easy to use for all user types	Non-Functional	High	Completed
NFR-03	User authentication and chat data must be secure and encrypted	Non-Functional	High	Completed
NFR-04	The app should function well across multiple Android devices and screen sizes	Non-Functional	Medium	Completed
NFR-05	Codebase should follow modular structure for maintainability and scalability (GetX for state management)	Non-Functional	Medium	Completed
NFR-06	App must handle low network conditions gracefully and sync missed messages	Non-Functional	Medium	In Progress

Table 5.1 - Requirement Matrix

6. Methodology

The Agile Model is a flexible and incremental software development methodology that promotes continuous planning, development, testing, and improvement throughout the project lifecycle. Instead of completing the project in one long cycle, the Agile approach breaks the development process into small, manageable sprints, allowing for rapid delivery and regular feedback.

6.1. Agile Model

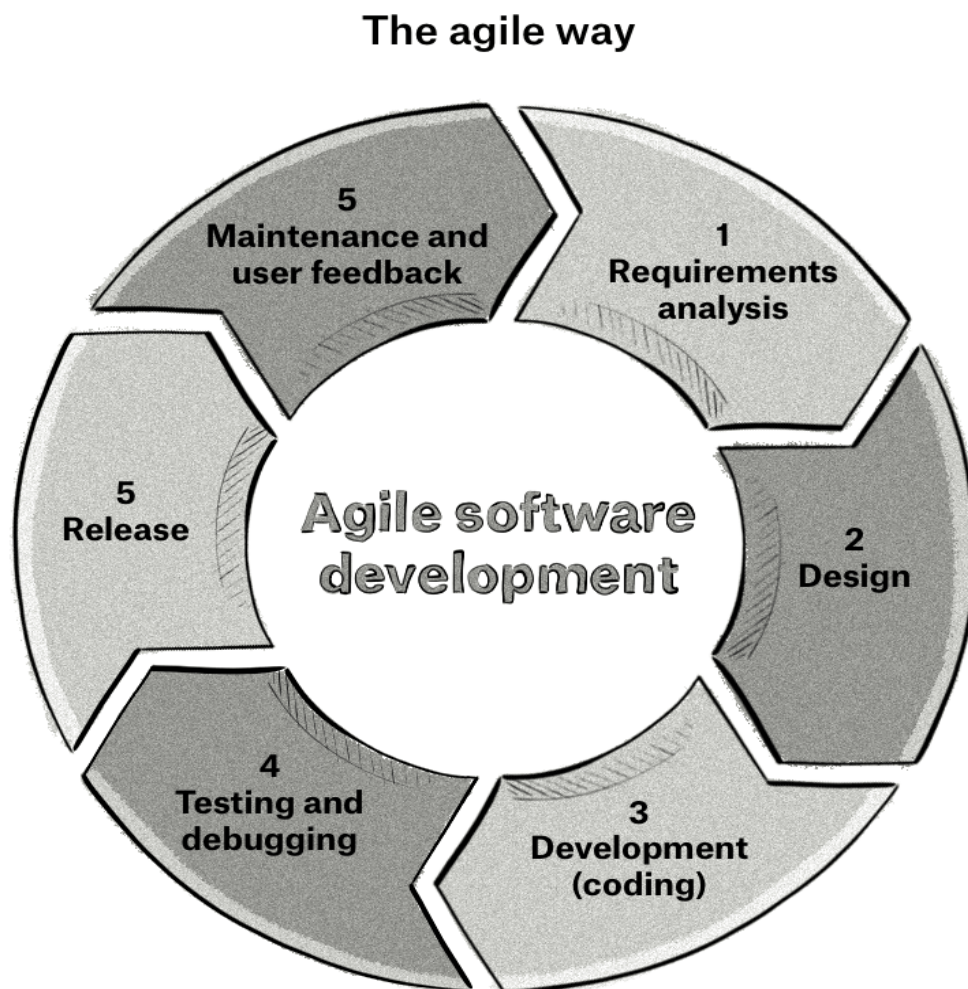


Figure 6.1- Agile Model

For the development of Real-Time Chat Box, we followed the Agile Model to ensure faster iterations, frequent testing, and constant communication among team members. This allowed us to adapt to changes, address issues early, and gradually integrate core

features like user authentication, real-time messaging, media sharing, and video/audio calling.

Each sprint in the Agile model followed this cycle:

- 1. Planning** – Define which feature or module to build (e.g., Login Screen, Chat Screen).
- 2. Design** – Create basic wireframes or UI mockups using Figma or paper sketches.
- 3. Development** – Implement logic using Flutter for the app, Node.js for backend, and Socket.IO for real-time features.
- 4. Testing** – Test each feature on devices/emulators to catch bugs early.
- 5. Review** – Team discussions and test-user feedback to guide changes.
- 6. Deployment** – Merge completed features into the main app progressively.

By following this approach, we ensured that the app remained responsive to feedback and delivered a user-centric communication tool for organizations.

6.1.1. Requirement Gathering:

We started the project by identifying the core functionalities required for a modern real-time messaging system. These included secure user registration and login, one-on-one and group chat support, real-time message delivery, file and media sharing, online/offline status, typing indicators, and audio/video calling features. Once the feature set was finalized, we decided on the technology stack:

- Flutter for developing the mobile application,
- Node.js + Express for the backend server and API layer,
- MongoDB for the database,
- Socket.IO for real-time WebSocket-based communication.

This selection allowed us to build a highly responsive and interactive app suitable for internal organizational use.

6.1.2. Analysis:

In this phase, we thoroughly analyzed the required features and grouped them into technical modules like authentication, messaging logic, media handling, and socket connection. We defined two main user roles: Admin and Normal User. Admins would have broader control, such as viewing all users and monitoring statuses, while normal users would focus on messaging and communication. We also outlined the backend logic for socket rooms, event broadcasting, and API routes for messages, user profiles, and file uploads. This helped in preparing a modular system for easier development and testing.

6.1.3. Design

We designed the application from both frontend and backend perspectives:

- For the mobile UI, we created wireframes and prototypes for screens like Login, Register, Chat List, Profile, and Chat Room using Flutter widgets and Material Design principles.
- On the backend, we structured the MongoDB collections for users, messages, chat groups, and media files.
- We designed the API structure using RESTful conventions and planned Socket.IO event handlers for real-time message exchange, typing indicators, and user status tracking.
- The codebase followed a modular GetX architecture for state management in Flutter, and folder-based MVC patterns on the backend.

6.1.4. Coding:

We followed an iterative sprint-based development approach to implement features incrementally:

- The backend was developed using Node.js with Express, handling APIs for user authentication, message storage, profile updates, and socket events.
- The mobile frontend was built using Flutter, implementing pages such as onboarding, login/register, home screen, chat UI, media picker, and profile.
- Socket.IO was integrated to support real-time messaging, typing indicators, seen status, and online user tracking

- Additional features like audio/video calls were prototyped using WebRTC and integrated gradually.

6.1.5. Testing

Each feature was manually tested after implementation using physical Android devices and emulators:

- We tested authentication, chat synchronization, socket events, emoji picker, media upload, file attachment, and typing/seen indicators.
- Bugs related to scroll behavior, offline storage, and socket reconnects were addressed.
- Testing included real-world use cases like slow networks, simultaneous logins, and media sharing to ensure stability.
- UI responsiveness, theme switching (light/dark), and user experience were closely evaluated and improved during each sprint.

6.1.6. Implementation

After completing and validating all features, we compiled the app using Flutter's build system and generated an APK for internal distribution. The backend was hosted on a local server for LAN-based real-time testing between mobile and laptop. The MongoDB database was hosted using MongoDB Atlas during the final phases to ensure data persistence across devices. The real-time chat system was deployed in a controlled environment to simulate realistic usage among peers for demo and feedback purposes.

6.1.7. Review

In the review phase, we gathered feedback from team members and test users to refine the system. Based on their inputs, we made UI adjustments (such as smoother chat transitions and auto-scroll behavior), improved socket reconnect logic, and enhanced message status indicators. We also optimized the emoji picker and file handling modules for a better user experience. Version control was maintained using GitHub, with regular commits, issue tracking, and code reviews to ensure the project stayed on track and collaborative.

6.2. Work Assigned

We are a group of 3 students involved in the development of the final year project and here is the work division chart among us:

S.N.	Name	Work Assigned	Remarks
1	Mausam Pariyar	Week 1: Flutter Project Setup, UI Planning Week 2: Authentication UI, GetX Setup Week 3: Chat Screen Design, UI for Typing/Status Week 4: Message Controller Logic, Emoji Picker, Reactions Week 5: Media Picker, File Sharing, Attachment Feature Week 6: Audio/Video Call Integration, Theming System Week 7: Final UI Polishing, Testing on Devices	Week 1: Satisfied Week 2: Satisfied Week 3: Refinement Week 4: Satisfied Week 5: Satisfied Week 6: Refinement Week 7: Satisfied
2	Sagar Poudel	Week 1: Node.js Setup, API Route Planning Week 2: User Auth API (Login/Register), JWT Handling Week 3: MongoDB Models (Users, Messages, Groups) Week 4: Message APIs, Seen/Delivered Logic Week 5: WebSocket Events for Typing, Online Status Week 6: Group Chat and Call Event Routing Week 7: Backend Optimization, Deployment Testing	Week 1: Satisfied Week 2: Satisfied Week 3: Satisfied Week 4: Refinement Week 5: Satisfied Week 6: Refinement Week 7: Satisfied
3	Ashish Gurung	Week 1: MongoDB Atlas Setup, Initial Schema Draft Week 2: Assist Backend with Mongoose Models Week 3: Chat Data Schema and Index	Week 1: Satisfied Week 2: Satisfied Week 3: Refinement

		Optimization	
		Week 4: Data Security, Token and Session Config	Week 4: Satisfied
		Week 5: Report Drafting	Week 5: Satisfied
		Week 6: Documentation (SDLC, Requirement Matrix)	Week 6: Satisfied
		Week 7: Final Report Compilation, Presentation Slides	Week 7: Satisfied

Table 6.1 – Worked Assigned

7. System Design

System design involves architecting the overall structure of the system, including how various modules interact, how data flows between them, and how users interact with the system. For **Real-Time Chat Box**, key design diagrams include the **Entity-Relationship Diagram (ERD)**, **Data Flow Diagrams (DFD Level 0, 1, 2)**, **UML Class Diagram**, and **Use Case Diagram**.

7.1. ER Diagram

The Entity-Relationship (ER) diagram helps in designing the database by identifying key entities, their attributes, and relationships. In our chat application, the main entities include **User**, **Message**, **Chat Room**, and **Media File**.

Key Entities:

- **User**: Contains attributes like `userId`, `fullName`, `email`, `profilePic`, `isAdmin`.
- **Message**: Includes `messageId`, `senderId`, `receiverId/chatRoomId`, `content`, `timestamp`, `status` (sent/delivered/seen).
- **ChatRoom**: Defines group or private chat with fields like `chatRoomId`, `members`, `type` (private/group).
- **Media**: Stores file attachments with `mediaId`, `fileUrl`, `type`, and `linkedMessageId`.

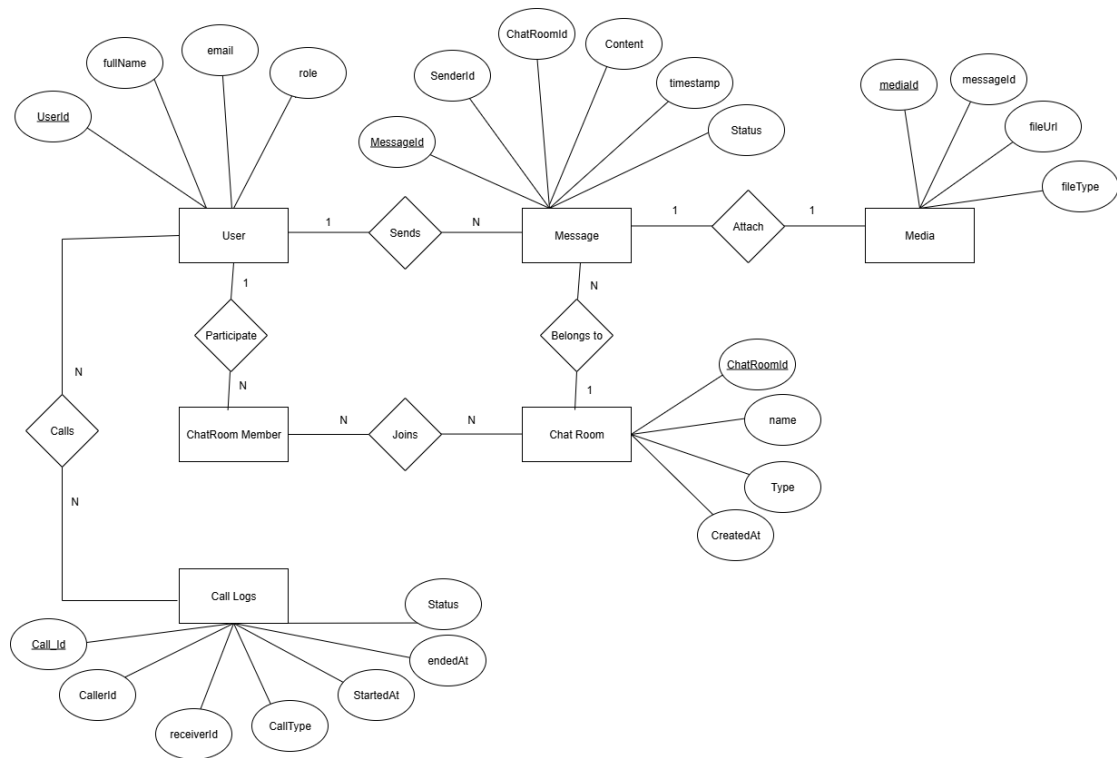


Figure 7.1 - ER Diagram

7.2. Dataflow Diagram

Data Flow Diagrams illustrate how data moves through the system and how various modules interact.

DFD Level 0 – Overview

At this level, the system is viewed as a single process:

- Users login/register
- Messages are sent and received in real-time
- Files/media are shared
- Admin views all user interactions

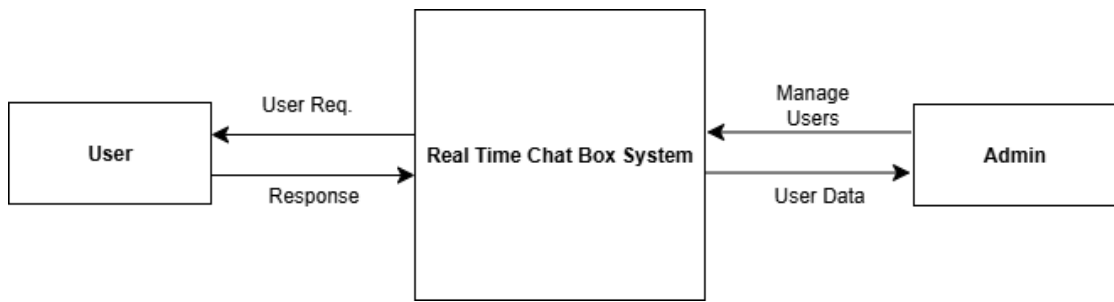


Figure 7.2 - DFD Level 0

DFD Level 1 – Module Breakdown

Breakdown of system into smaller processes:

- **Authenticate User:** Validate credentials via API
- **Send/Receive Messages:** Via Socket.IO
- **Store/Fetch Messages:** Using MongoDB
- **Media Upload:** Store in file system / cloud and link to message

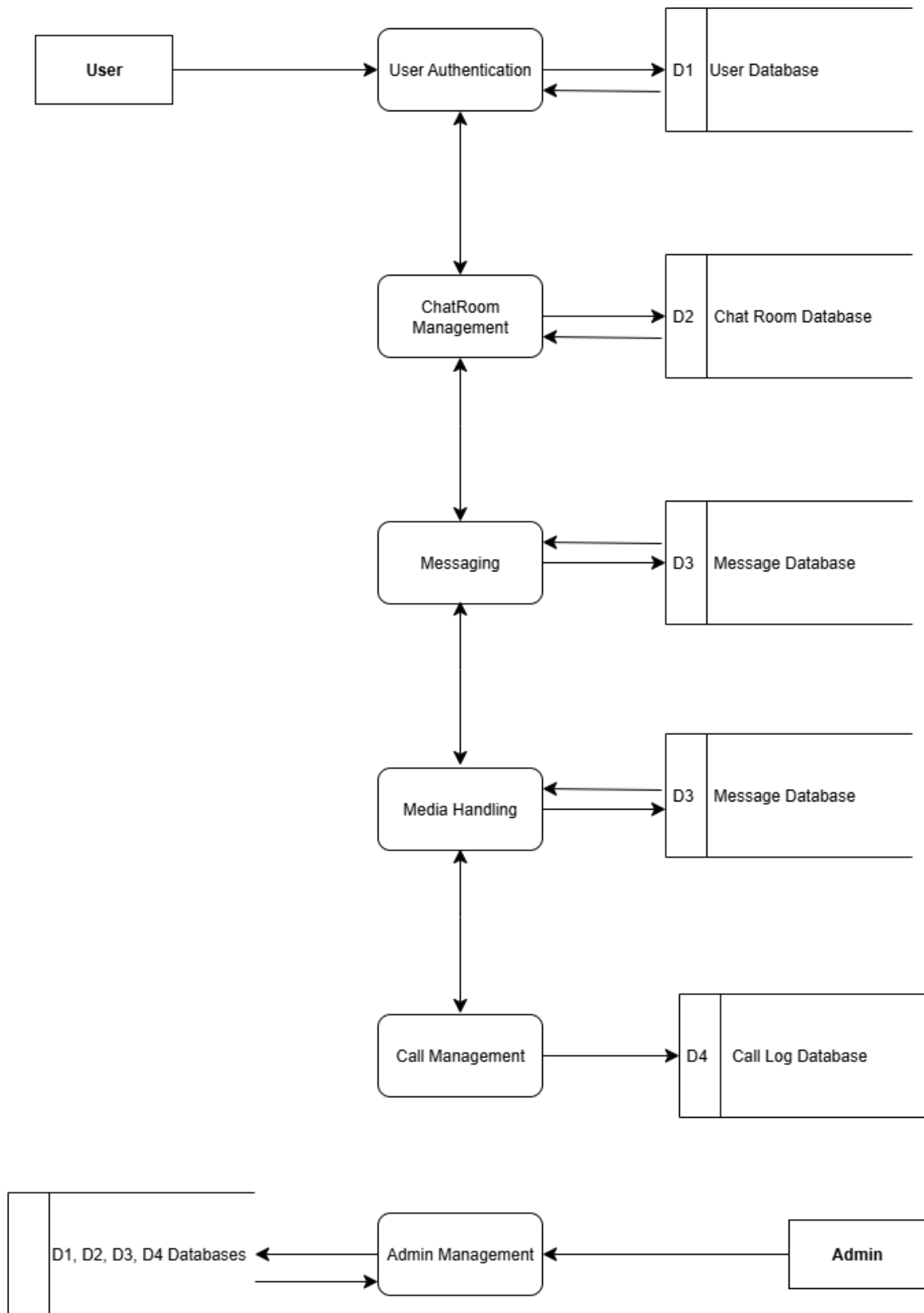


Figure 7.3 - DFD Level 1

DFD Level 2 – From User Perspective

- User opens app → Authenticates → Navigates to chat list
- Selects chat → Sends message (text or file)
- Receives messages in real-time → Views chat history
- Can make a voice/video call → Ends session

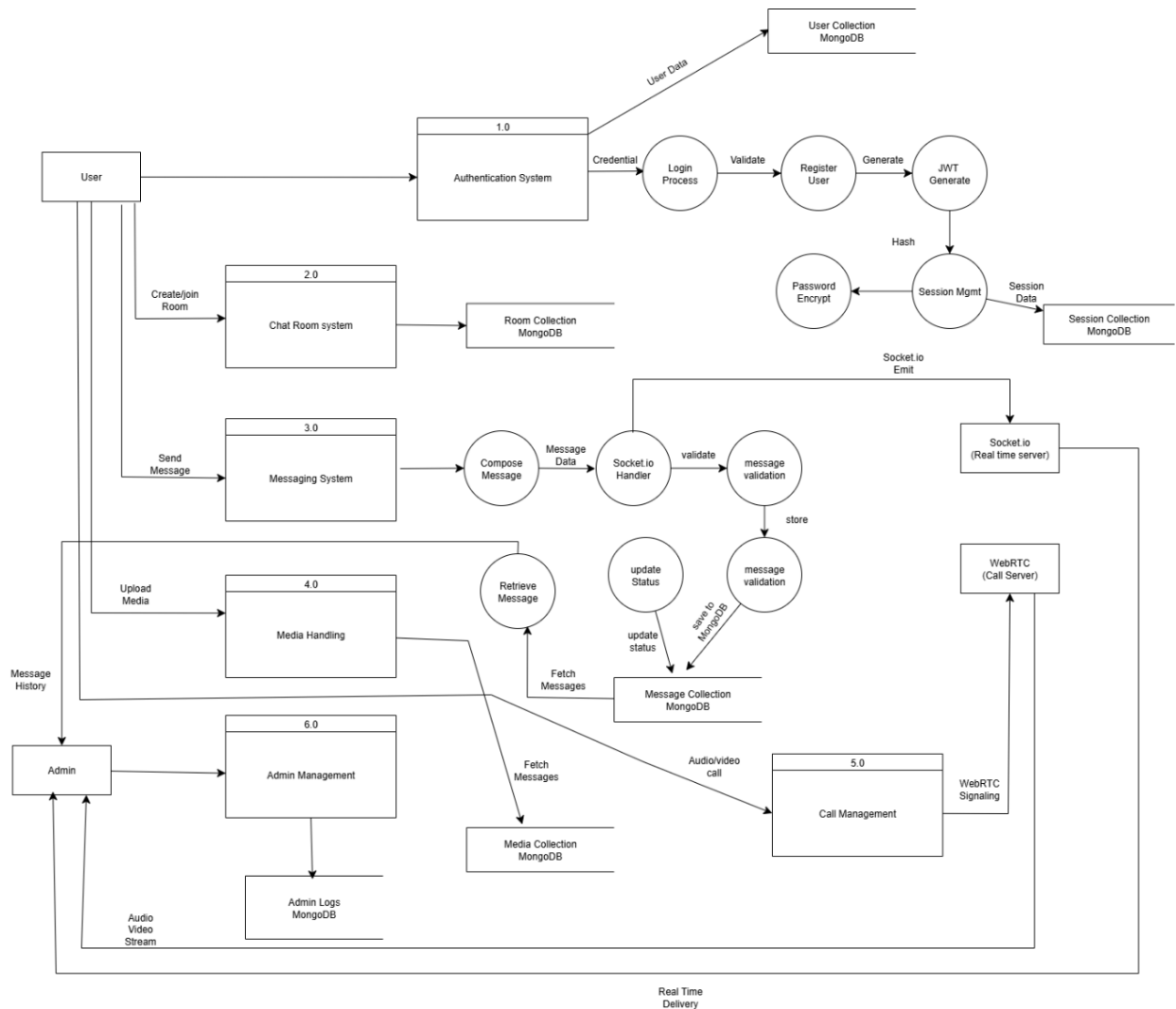


Figure 7.4 - DFD Level 2

7.3. UML Diagram

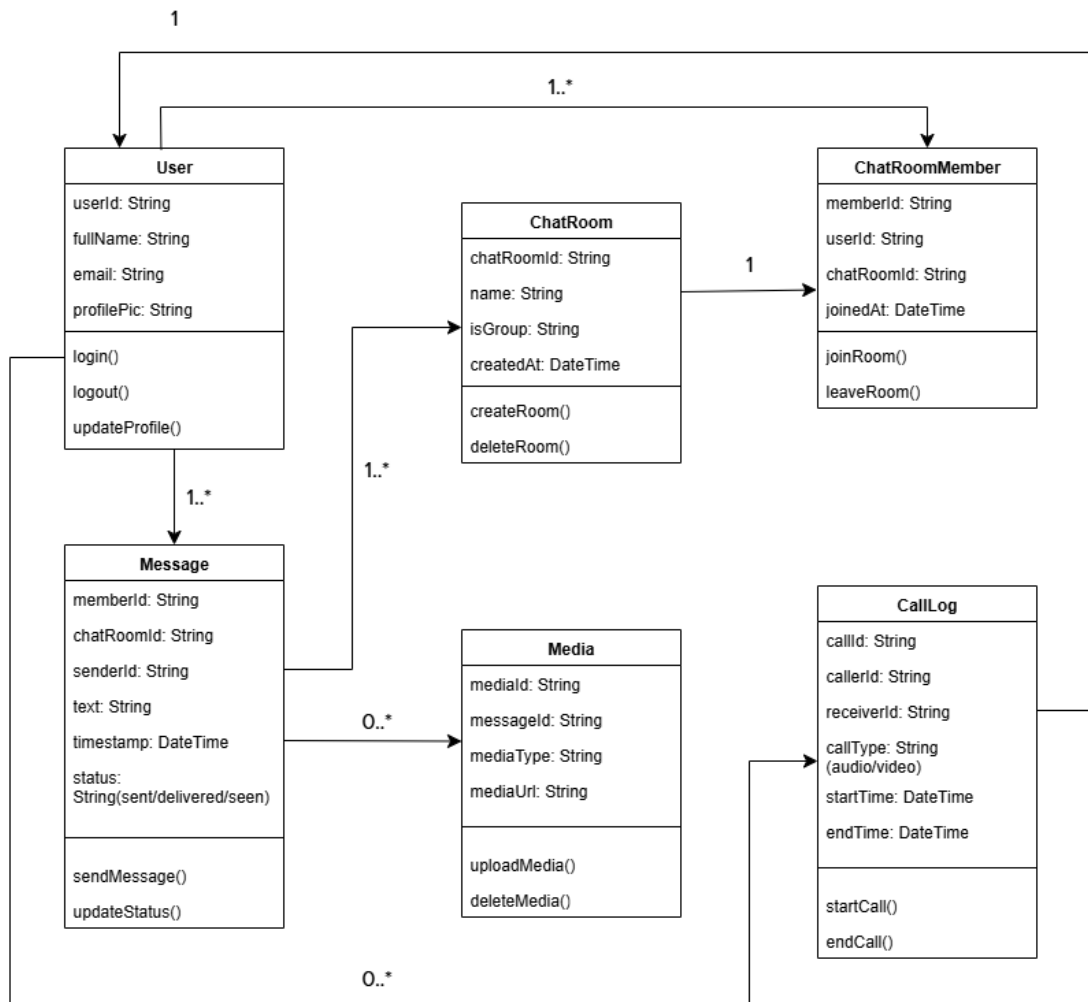


Figure 7.5 - UML Diagram

8. Gantt Chart

The Gantt chart below shows the schedule planned for “Real-Time Chat Box”.

Task Name	May 1	May 15	June 1	June 15	July 1
Planning					
Research					
Design					
Coding					
Debugging					
Implementation					
Documentation					

Table 8.1 - Gantt Chart

*Note: The Duration of one interval is of 15 days.

9. Development Process

The development of the Real-Time Chat Box application was carried out using the Agile Methodology in conjunction with an Iterative Development Model. This approach allowed the team to develop the application incrementally in multiple cycles of planning, implementation, testing, and refinement. Agile was selected due to its flexibility, focus on collaboration, and ability to adapt to changing requirements throughout the software development life cycle.

Iterative Planning and Execution

The project was divided into weekly iterations (sprints), with each sprint focused on delivering a specific set of features or improvements. At the beginning of each iteration, sprint planning sessions were conducted to:

- Identify the tasks to be completed,
- Assign responsibilities based on team members' expertise,
- Estimate the time and effort required for each task.

Progress was tracked throughout the week, and the sprint concluded with a review and minor refinement based on testing feedback. This continuous cycle ensured the steady development of core functionalities and early detection of issues.

Frontend and Backend Development

Development was carried out in parallel across different parts of the application, including mobile, backend, and database components.

Flutter Mobile App (Frontend)

- Built using Flutter to support cross-platform compatibility and a consistent user interface
- Included screens for user authentication, chat UI, group chat, media sharing, and profile management
- Integrated WebSocket communication using Socket.IO for real-time message delivery

Web and Backend (API Layer)

- Developed using Node.js and Express.js to handle backend logic and API endpoints
- Implemented RESTful APIs for user management, chat handling, media uploads, and call logging
- Socket.IO was used for real-time communication, enabling instant delivery of messages and updates

MongoDB (Database)

- MongoDB was used as the primary NoSQL database
- Collections were designed for users, messages, chatrooms, media, and call logs
- Relationships were managed to support scalability, fast queries, and real-time synchronization

Key Features Implemented

- User registration and login functionality
- Real-time one-to-one and group messaging
- Audio and video call logging
- Media sharing (images, files)
- Chatroom creation and membership handling
- Profile updates and user status tracking

Testing and Version Control

Manual testing was performed on Android devices after each development phase to ensure:

- Smooth UI and performance
- Real-time message sync
- Proper media handling and error management

Version control was managed using GitHub:

- Code was pushed regularly using Git and organized into branches
- Pull requests were used to review and merge features
- GitHub issues helped in bug tracking and task assignments

Team Collaboration

The team worked closely with clearly defined roles:

- **Mausam Pariyar** – Flutter mobile app development (UI components, messaging logic, chatroom management)
- **Sagar Poudel** – Web and backend development (API, Socket.IO, authentication, chatroom services)
- **Ashish Gurung** – MongoDB database design and schema creation, overall documentation and report writing

Communication and coordination were maintained through collaborative tools such as GitHub and group chat platforms. Weekly standups and code reviews ensured steady progress and alignment among all team members.

10. Testing

Testing is a critical step in the software development life cycle to ensure that the application meets the desired functionality, performance, and reliability. For our **Real-Time Chat Box Application**, testing was performed iteratively throughout development, aligning with the Agile methodology. This ensured that bugs were identified and resolved early, and that the chat features worked as expected in various scenarios.

Project Name: Real-Time Chat Box (Test Case No: -1)
Test Case IDs: TC01, TC02, TC03, TC04
Test Priority: High
Module Name: Authentication Module
Test Title: User Registration, Login, and Forgot Password
Test Executed By: Mausam Pariyar
Pre-condition: User must have access to registration and login screen with a valid email and password.
Description: Verifies registration, login, and password recovery flows with valid and invalid inputs.

Test Case No: 01 – User Authentication

Test Case ID	Description	Test Steps	Expected Result	Actual Result	Status
TC01	User can register	Fill registration form and submit	User is created successfully	As expected	Pass
TC02	User can log in	Enter valid credentials	Redirect to home screen	As expected	Pass
TC03	Invalid login	Enter incorrect credentials	Error message shown	As expected	Pass

TC04	Forgot password	Click forgot password and follow reset steps	Password reset success	As expected	Pass
------	-----------------	--	------------------------	-------------	------

Table 10.1 Test case 1

Test Case No: - 2
Test Case IDs: TC05, TC06, TC07
Test Priority: High
Module Name: Chatroom Management
Test Title: Chatroom Creation, Join Via Invite, and Leave Chatroom
Test Executed By: Mausam Pariyar
Pre-condition: User must be authenticated and able to access chatroom interface.
Description: Validates creation of chatrooms, joining via invite link, and leaving functionality.

Test Case No: 02 – Chatroom Creation & Joining

Test Case ID	Description	Test Steps	Expected Result	Actual Result	Status
TC05	Create a new chatroom	Enter chatroom name and tap create	Chatroom created and visible	As expected	Pass
TC06	Join chatroom via invite	Click on invite link	User joined the chatroom	As expected	Pass
TC07	Leave chatroom	Click leave button inside chatroom	User is removed from chatroom	As expected	Pass

Table 10.2 Test case 2

Test Case No: - 3
Test Case IDs: TC08, TC09, TC10, TC11
Test Priority: High
Module Name: Messaging Core
Test Title: Sending, Receiving, Chronological Order, Offline Sync
Test Executed By: Ashish Gurung
Pre-condition: Users should be in same chatroom and actively connected via socket.
Description: Tests message delivery, ordering, and syncing when offline and online.

Test Case No: 03 – Sending & Receiving Messages

Test Case ID	Description	Test Steps	Expected Result	Actual Result	Status
TC08	Send message	Enter and send a message in a chatroom	Message appears instantly	As expected	Pass
TC09	Receive message	User B receives the sent message	Message delivered real-time	As expected	Pass
TC10	Message order	Send multiple messages quickly	Order maintained chronologically	As expected	Pass
TC11	Offline message sync	Send message while User B is offline, then reconnect	Message syncs on reconnect	As expected	Pass

Table 10.3 Test case 3

Test Case No: - 4
Test Case IDs: TC12
Test Priority: Medium
Module Name: Messaging UX Enhancement
Test Title: Typing Indicator Visibility
Test Executed By: Mausam Pariyar
Pre-condition: Both users online in same chatroom; WebSocket should be active.
Description: Ensures typing indicator appears and disappears correctly in real time.

Test Case No: 04 – Typing Indicator

Test Case ID	Description	Test Steps	Expected Result	Actual Result	Status
TC12	Show typing indicator	Start typing a message in chatroom	Other users see 'User is typing...'	As expected	Pass

Table 10.4 Test case 4

Test Case No: - 5
Test Case IDs: TC13, TC14
Test Priority: Medium
Module Name: Message Status Tracking
Test Title: Seen Status in One-to-One and Group Chats
Test Executed By: Sagar Poudel
Pre-condition: Messages must be sent; recipients must be online or open the chat.
Description: Confirms whether the seen/read receipt is triggered and displayed properly.

Test Case No: 05 – Message Seen Status

Test Case ID	Description	Test Steps	Expected Result	Actual Result	Status
TC13	Mark message as seen	Open chatroom and view new messages	'Seen' label appears below message	As expected	Pass
TC14	Seen by multiple users	All group members view the message	Seen status shows for all users	As expected	Pass

Table 10.5 Test case 5

Test Case No: - 6
Test Case IDs: TC15, TC16
Test Priority: Medium
Module Name: Messaging UX Enhancement
Test Title: Reply to Messages and Swipe to Reply
Test Executed By: Ashish Gurung
Pre-condition: Messages must already exist in chatroom.
Description: Verifies emoji reaction functionality and reply mechanics using swipe gestures.

Test Case No: 06 – Reactions & Replies

Test Case ID	Description	Test Steps	Expected Result	Actual Result	Status
TC15	React to message	Long-press message and select emoji	Emoji reaction appears below message	As expected	Pass
TC16	Reply to message	Swipe message to reply	Reply is shown above input field	As expected	Pass

Table 10.6 Test case 6

Test Case No: - 7
Test Case IDs: TC17, TC18
Test Priority: High
Module Name: Media Handling
Test Title: Sending Media Files and Downloading Attachments
Test Executed By: Mausam Pariyar
Pre-condition: File Picker accessible; users must be in an active chatroom.
Description: Checks upload, send and download features for images, videos, and documents.

Test Case No: 07 – File Sharing

Test Case ID	Description	Test Steps	Expected Result	Actual Result	Status
TC17	Send image	Attach and send image	File appears in chat	As expected	Pass
TC18	Download attachment	Tap image/file in chat	Opens/downloads file	As expected	Pass

Table 10.7 Test case 7

Test Case No: - 8
Test Case IDs: TC19, TC20, TC21
Test Priority: High
Module Name: Audio/Video Call
Test Title: Initiating Calls and Logging Call History
Test Executed By: Mausam Pariyar & Sagar Poudel
Pre-condition: Both users online with socket connection active and calling feature integrated.
Description: Tests audio/video call initiation ringing flow, and log persistence.

Test Case No: 08 – Call Logs & Audio/Video Call

Test Case ID	Description	Test Steps	Expected Result	Actual Result	Status
TC19	Start audio call	Tap audio call icon in chatroom	Call initiates	As expected	Pass
TC20	Start video call	Tap video call icon	Video call interface opens	As expected	Pass
TC21	Call log update	End call and view call log	Call duration/status recorded	As expected	Pass

Table 10.8 Test case 8

Test Case No: - 9
Test Case IDs: TC22
Test Priority: Medium
Module Name: Session Management
Test Title: Logout and Session Cleanup
Test Executed By: Ashish Gurung
Pre-condition: User must be authenticated and inside the app to perform logout.
Description: Ensures session data is cleared and user is redirected properly on logout.

Test Case No: 09 – Logout & Session Handling

Test Case ID	Description	Test Steps	Expected Result	Actual Result	Status
TC22	Logout	Tap logout in settings	Session ends and redirect to login screen	As expected	Pass

Table 10.9 Test case 9

11. Deliverables

The deliverables of this project are as follows:

- **User Authentication System** for secure login, registration, and role-based access control for both users and administrators.
- **Real-Time Messaging Module** that enables users to send and receive messages instantly using WebSockets (Socket.IO), including seen status, typing indicators, and emoji support.
- **Audio and Video Calling Functionality** allowing users to make peer-to-peer audio/video calls with connection status, duration tracking, and call logging features.
- **Media Sharing Support** enabling users to upload and view files such as images, videos, and documents within the chat interface.
- **Profile Management Interface** where users can view and update their personal details, including profile pictures, email, and status.
- **Customizable Theme Options** that allow users to personalize the appearance of the chat interface including color themes, fonts, and chat bubble styles.
- **Secure Data Storage** using MongoDB and JWT-based token authentication to ensure encrypted communication and safe data handling across the application.
- **User-Friendly Interface** designed in Flutter to deliver a seamless chat experience with responsive UI components and accessible navigation for modern users.

12. Project Result

After completing the development and testing phases, the Real-Time Chat Box project has successfully achieved its primary goals. The system now functions as a responsive and secure communication platform, enabling real-time text messaging, media sharing, and audio/video calls. Designed with flexibility and scalability in mind, the application delivers a seamless experience across devices and sets the foundation for future enhancements.

12.1. What are the functions achieved by our project?

The Real-Time Chat Box offers a comprehensive suite of communication features:

- **Secure User Authentication**

The system supports role-based registration and login, providing secure access to users via token-based authentication and email verification.

- **Real-Time Messaging System**

Users can exchange messages instantly with seen/delivered status, typing indicators, emoji support, and message reactions, powered by WebSocket (Socket.IO) communication.

- **Audio/Video Calling Feature**

Integrated audio and video calling capabilities allow users to initiate peer-to-peer calls with clear streaming and live status updates.

- **Media Sharing & Attachments**

Users can send images, videos, documents, and other media files within chats, with previews and download options.

- **Customizable Theme Support**

The app provides theme customization features, allowing users to personalize the UI, including font styles, chat bubble colors, and background modes.

- **Profile and Settings Management**

Each user can update their profile picture, name, and status, and access various chat settings.

12.2. What problems have been solved by the project?

The Real-Time Chat Box addresses key communication challenges faced by users and developers:

- **Lack of Unified Real-Time Communication**

Many platforms lacked an all-in-one solution combining text, media, and video communication. This app offers an integrated chat experience with real-time responsiveness.

- **Scalability and Flexibility Issues**

Traditional chat systems often faced limitations in group handling, media exchange, or socket-based performance. This solution introduces scalable room-based messaging and clean data architecture using MongoDB and Socket.IO.

- **User Experience and Personalization Gaps**

The lack of customization in many apps led to monotonous user experiences. Our app offers theme customization and personalized profiles, enhancing long-term engagement.

- **Security Risks in Communication Apps**

The app uses token-based authentication, secure media handling, and role-based access to ensure secure interaction between users.

By solving these problems, the Real-Time Chat Box provides a dependable and user-centric messaging platform that promotes instant, reliable, and personalized communication.

13. Future Enhancements

To ensure that the Real-Time Chat Box continues to evolve alongside user expectations and technological trends, the following enhancements are proposed for future versions. These improvements aim to boost communication efficiency, strengthen security, and provide a more engaging and personalized user experience.

1. Advanced Communication Features

- **Video and Voice Calling**

Integration of WebRTC to support real-time audio and video calls directly within the chat interface.

- **Screen Sharing**

Enable users to share their screens during conversations, useful for meetings, collaboration, and support.

- **Message Reactions**

Allow users to respond to specific messages using emojis to improve expressiveness.

2. Improved User Experience

- **Customizable Themes**

Offer a selection of UI color schemes and layout styles, allowing users to personalize the app's appearance.

- **Push Notifications**

Notify users in real-time when they receive new messages, even when the app is minimized or closed.

- **Message Pinning**

Let users pin important messages or conversations to the top of the chat for easy access.

- **Chat Search Functionality**

Add a message search feature to locate specific content using keywords or filters.

3. Security and Moderation

- **Self-Destructing Messages**

Introduce ephemeral messaging where messages automatically disappear after a set period.

- **AI-Powered Spam Detection**

Implement machine learning to identify and filter out spam or inappropriate content automatically.

4. Scalability and Integrations

- **Channel-Based Group Chats**

Create Slack/Discord-style channels within groups for organized topic-based conversations.

- **CRM and Helpdesk Integration**

Extend the chat system to support business use cases through integration with CRM platforms and customer service tools.

- **Offline Messaging Support**

Allow users to compose and queue messages when offline, which will be sent automatically once connectivity is restored.

These enhancements will significantly improve the platform's performance, usability, and versatility, making the Real-Time Chat Box adaptable for both personal and professional use cases.

14. Conclusion

The Real-Time Chat Box is a powerful and user-friendly communication platform developed to support fast, secure, and real-time messaging. Designed as part of the BCA curriculum under Pokhara University, this project focuses on solving key challenges such as delayed message delivery, complex user interfaces, and weak security in existing messaging apps. By incorporating technologies like Socket.IO for real-time communication and a modern frontend framework, the app ensures smooth and responsive performance on mobile devices.

One of the major strengths of the system lies in its secure authentication mechanism, allowing only verified users to access chat features. Users can send text messages, share media files like images and videos, and manage their profile within a clean and intuitive interface. The app also follows a modular architecture, making it easier to maintain and scale over time. Its design emphasizes ease of use, performance optimization, and responsiveness, making it suitable for users of all technical backgrounds.

Additionally, the Real-Time Chat Box addresses real-world communication problems by eliminating delays, simplifying the chat process, and providing a reliable platform for both individual and group conversations. It promotes collaboration, improves user experience, and ensures data protection, making it a practical solution for educational institutions, businesses, and everyday use.

In summary, the Real-Time Chat Box is not just a chat application, it is a communication system built with modern tools, thoughtful design, and scalable architecture. It serves as a valuable project with strong real-world application potential, fulfilling both academic goals and future expansion opportunities.

15. Reference

- Ahmad, I., Yousaf, M. M., & Khan, M. A. (2019). Design and development of secure chat application using end-to-end encryption. *International Journal of Computer Applications*, 178(15), 8–13. Retrieved from: <https://doi.org/10.5120/ijca2019918920>
- Bhat, A., & Ahmed, M. (2020). Implementation of a real-time chat application using Firebase. *International Journal of Engineering Research & Technology (IJERT)*, 9(5), 273–276.
- Discord. (n.d.). Your place to talk and hang out. Retrieved July 9, 2025, from <https://discord.com> Retrieved from: <https://discord.com>
- Facebook. (n.d.). Messenger – Text, voice, and video chat. Retrieved July 9, 2025, from <https://www.messenger.com> Retrieved from: <https://www.messenger.com>
- Google. (2021). Messages by Google – Simple, helpful messaging. Retrieved from <https://messages.google.com> Retrieved from: <https://messages.google.com>
- Khan, F., Mantri, N., Rajput, S., Dhakane, D., & Padiya, P. (2020). Anonymous decentralized ephemeral chat application using Interplanetary File System. *ITM Web of Conferences*, 32, 02004. Retrieved from: <https://doi.org/10.1051/itmconf/20203202004>
- Nepal Telecom. (2022, August 10). Top messaging apps in Nepal: Facebook Messenger, Viber and more. Nepali Telecom. Retrieved from: <https://www.nepalitelecom.com/top-messaging-apps-in-nepal>
- Patel, N., Patel, M., & Patel, H. (2017). Secure chat application using hybrid cryptography algorithms. *International Journal of Advanced Research in Computer Science*, 8(5), 276–279.
- Slack. (n.d.). Slack: Where work happens. Retrieved July 9, 2025, from <https://slack.com> Retrieved from: <https://slack.com>
- Statista. (2024). Most popular global mobile messenger apps as of January 2024, by number of monthly active users. Retrieved from: <https://www.statista.com/statistics/258749/most-popular-global-mobile-messenger-apps>

Telegram. (n.d.). Telegram Messenger. Retrieved July 9, 2025, from <https://telegram.org> Retrieved from: <https://telegram.org>

Thapa, R. (2023). End-to-end encrypted mobile chat application using Firebase and Flutter. *Journal of Emerging Technologies and Innovative Research (JETIR)*, 10(2), 101–110.

Viber. (n.d.). Free and secure calls and messages to anyone, anywhere. Retrieved July 9, 2025, from <https://www.viber.com> Retrieved from: <https://www.viber.com>

WhatsApp. (n.d.). Simple. Secure. Reliable messaging. Retrieved July 9, 2025, from <https://www.whatsapp.com> Retrieved from: <https://www.whatsapp.com>

Zhang, Y., & Yang, X. (2018). Real-time instant messaging app based on WebSocket. *Proceedings of the 2nd International Conference on Education, Management and Social Science*, 189–192.

16. Annex

This annex section provides supportive content that enhances the understanding and documentation of the Real-Time Chat Box project. It includes details of technologies used, team contributions, relevant tools, project resources, and supporting components that guided the development and completion of the system.

The project utilized modern technologies such as Flutter for mobile development, Node.js and Express.js for backend logic, MongoDB for scalable database storage, and Socket.IO for enabling real-time communication. Additional tools like GetX and GetStorage were used to manage state and local data storage efficiently within the Flutter app.

Team members contributed across different modules including frontend development, backend API creation, testing, and documentation. Version control was maintained through GitHub repositories, supporting organized and collaborative development.

Visual references such as UI screenshots (login screen, chat screen, user settings, etc.) are included to demonstrate the app's functionality. A glossary of technical terms is also presented to aid readers in understanding key concepts used in the system.

Together, these annex materials provide a complete overview of the project's technical backbone, development workflow, and implementation details.

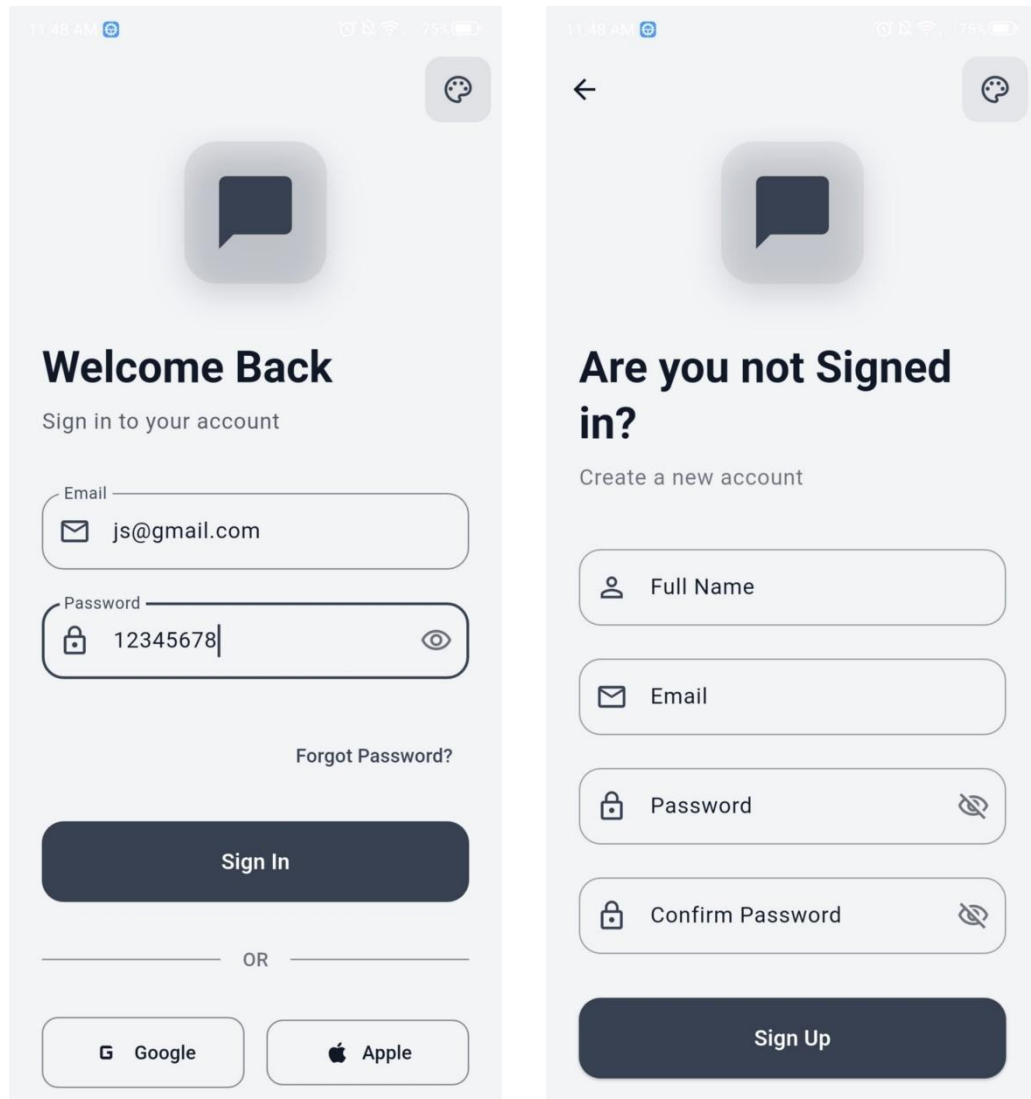


Figure 16.1 - Sign In and Sign Up Page

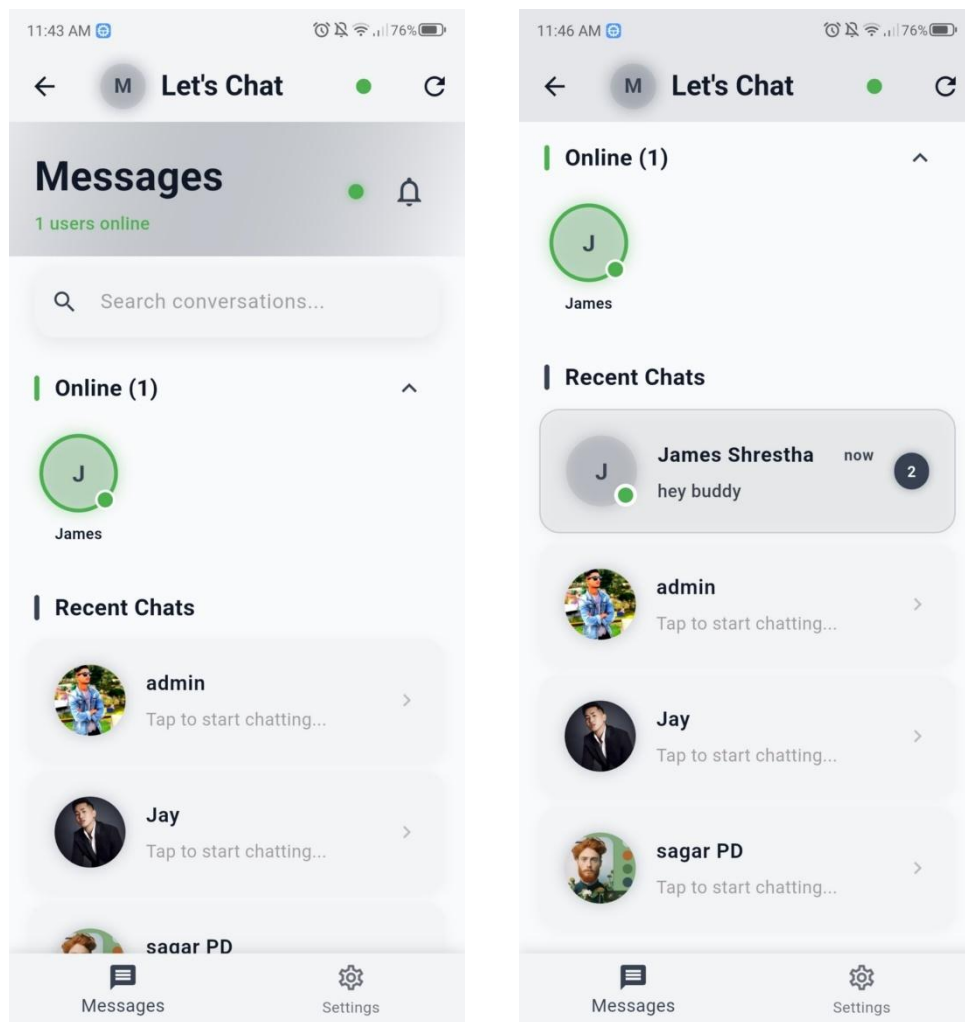


Figure 16.2 – Homepage

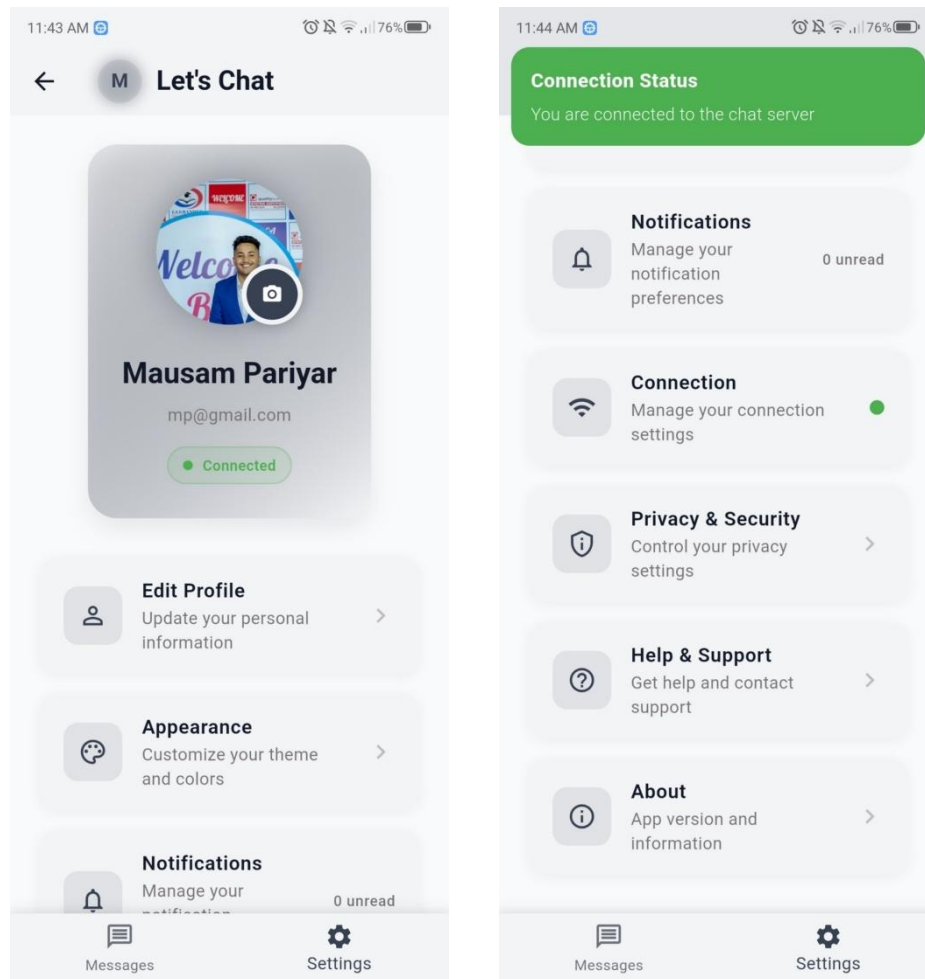


Figure 16.3 – Profile page and Other Settings

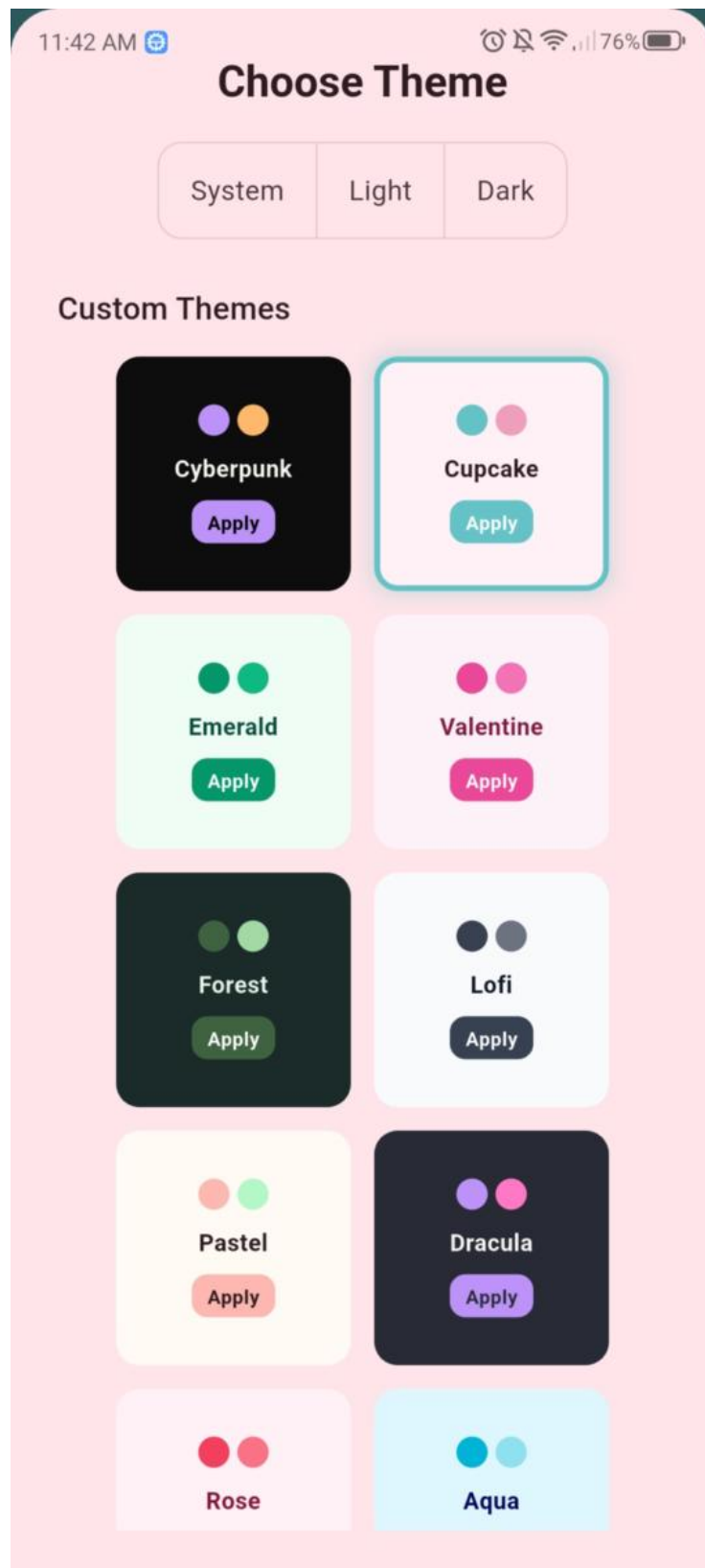


Figure 16.4 Custom Theme Palette

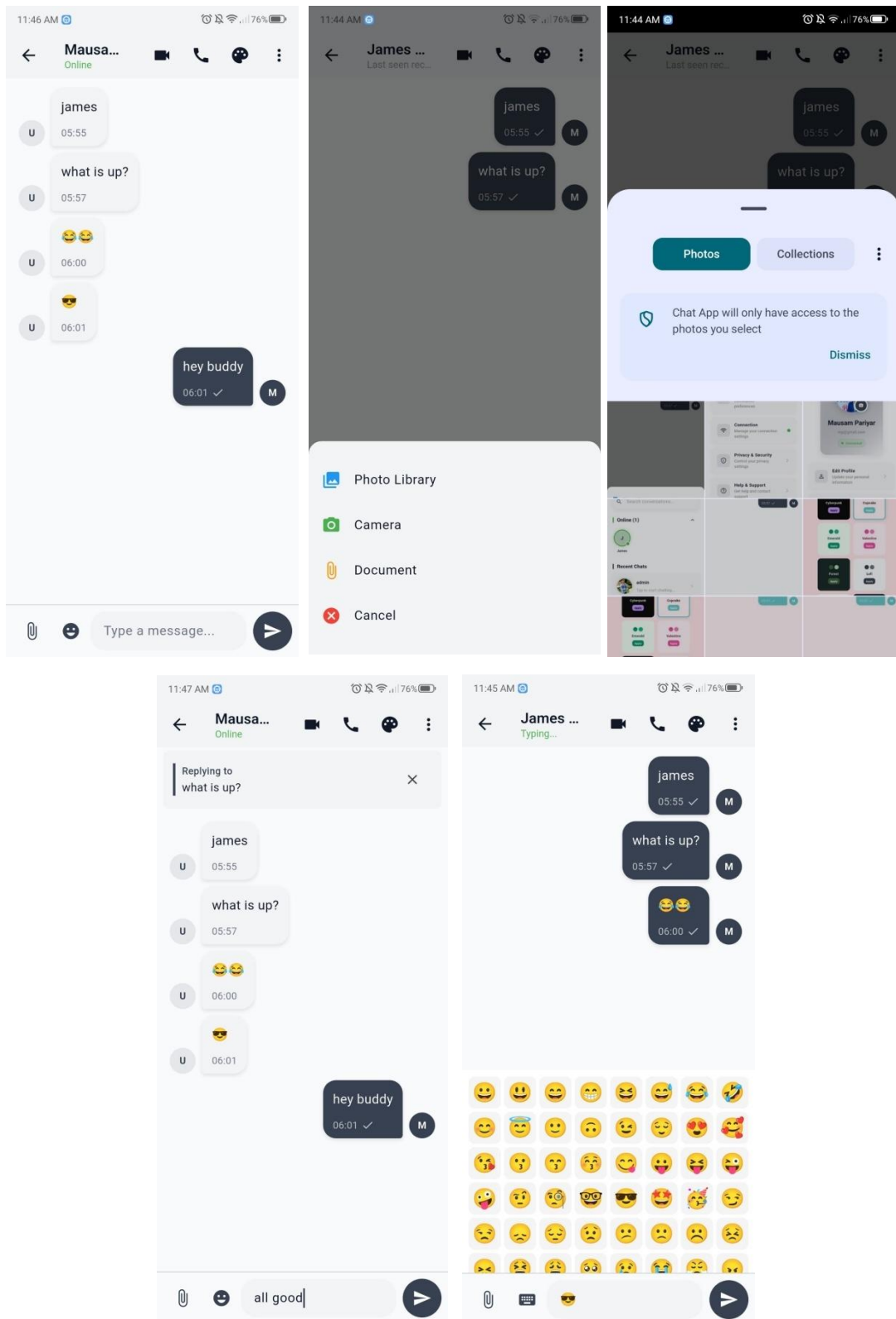


Figure 16.5 Chat Screen

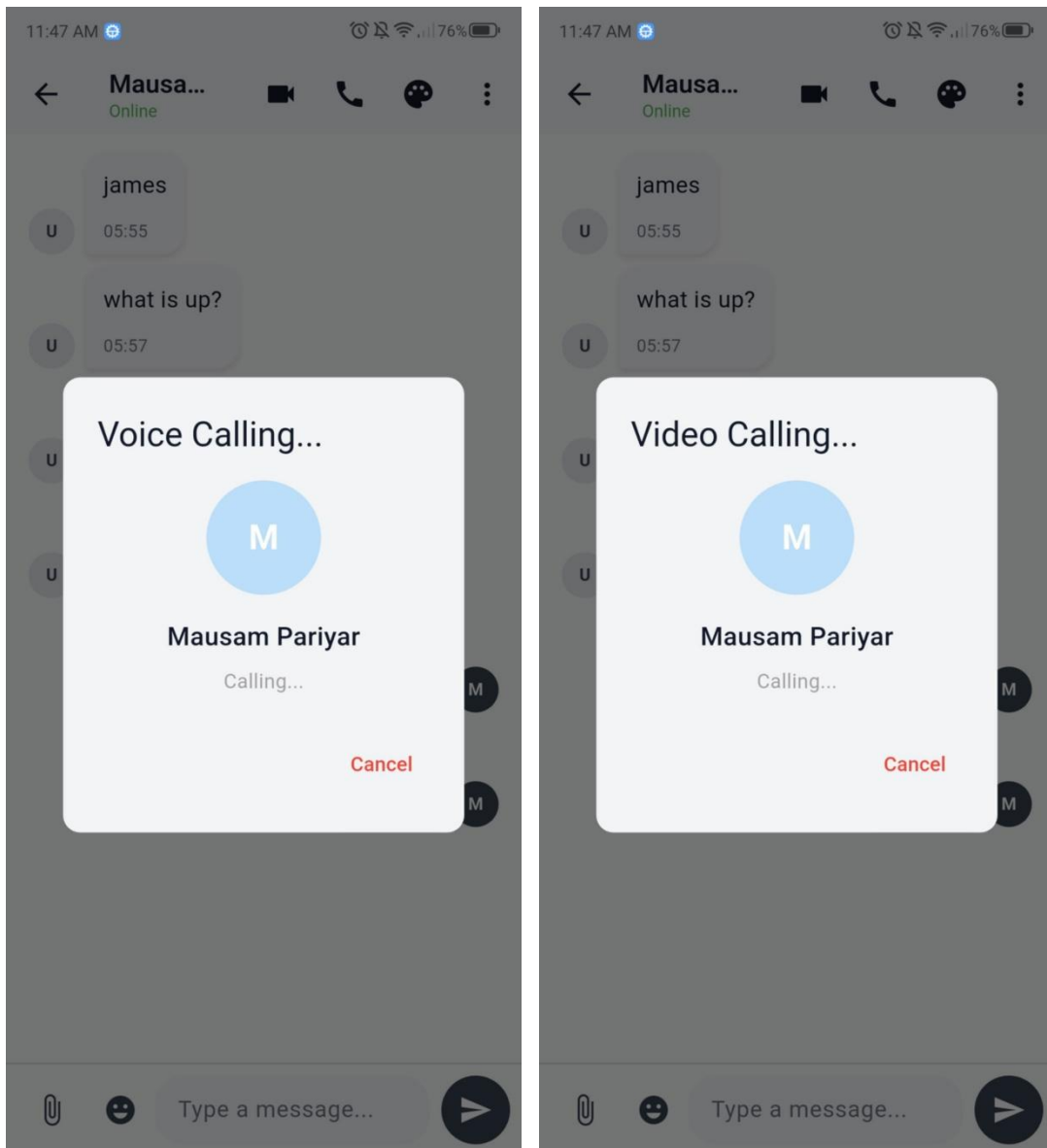


Figure 16.6 Call Screen